

# Transfer Attacks in Face Recognition

**MEF (Maximin Expected Flatness) — A Flatness-Seeking Transfer Attack**

Hiya Trehan

July 24 2026

Implementation → <https://github.com/Heya28/transferattacktInterns>

# Attacks Considered

## Vanilla baselines (existing in the pipeline):

- PGD (baseline)
- MI-FGSM (momentum-based)
- TI-FGSM (translation-invariant)
- SI-NI-FGSM (scale-invariant, Nesterov)
- MI-ADMIX-DI-TI (input-diversity + admix)

## The contribution:

- MEF (Maximin Expected Flatness) — a neighborhood-sampling, flatness-seeking momentum attack

*Reference: “Boosting Adversarial Transferability with Low-Cost Optimization via Maximin Expected Flatness” — IEEE T-IFS 2024*

# Why MEF? The Transfer Problem

## Definition & Nature

- Neighborhood-sampling momentum attack
- Two-level (inner + outer) optimization
- Bi-level, gradient-based,  $L^\infty$ -bounded
- Estimates expected loss over a region, not a single point

## Goal in Face Recognition

- Impersonation: maximize  $\cos(\text{adv}, \text{target})$
- Dodging: minimize  $\cos(\text{adv}, \text{self})$
- Success = threshold crossing on the victim model

## Basic Idea

- Plain gradient ascent finds sharp, surrogate-specific optima
- Sharp optima rarely survive on an unseen victim model
- MEF instead searches for flat regions of high attack success
- Flat regions are more likely to overlap across architectures

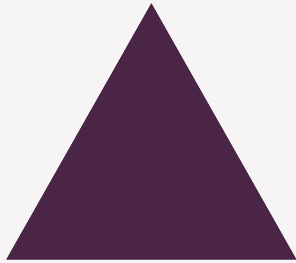
## Difference From Plain Momentum (MI-FGSM)

- MI-FGSM stabilizes direction over time, but still evaluates one point per step
- MEF evaluates 20 sampled neighbors per step and averages
- Each neighbor carries its own local momentum (PGIA)

# Continued → Why MEF? The Problem With Chasing a Single Point

Every transfer attack faces the same core problem:  
**a perturbation crafted against one model (the surrogate) rarely works as well against a different, unseen model (the victim) — because it was optimized against that surrogate's specific loss geometry.**

**A Sharp Optimum (what plain gradient ascent finds)**



loss / attack success

A narrow spike of high attack-success, unique to the surrogate's exact geometry. The victim's landscape looks different — this exact spike likely doesn't exist there. Result: high success on surrogate, breach fails on victim.

**A Flat Region (what MEF searches for)**



loss / attack success

A wide plateau of high attack-success. Small architectural differences between surrogate and victim are far less likely to knock you out of a wide plateau than a narrow spike. Result: more likely to hold up on an unseen victim.

# Working of MEF Attack

- 1 Initialize adversarial image as original input; set outer momentum  $g=0$
- 2 For each of  $T=5$  outer iterations, run an inner loop of  $K=20$  samples:
  - 3 Draw a random neighbor within radius  $\gamma \cdot \epsilon$  of the current point
  - 4 Nudge it toward that sample's own running momentum (PGIA), scaled by  $\kappa$
  - 5 Compute the gradient of the attack loss at that nudged point
- 6 Average all 20 inner-loop gradients — this is the expected-loss estimate
- 7 Update each sample's personal momentum from its own gradient
- 8 Blend the averaged gradient with outer momentum ( $\mu=0.5$ ) and normalize
- 9 Update image using sign of the blended gradient
- 10 Project image into  $\epsilon$ -bounded region; clip to valid pixel range
- 11 Return final adversarial image after  $T$  iterations

# Attack Implementation

## Pseudocode

```
Input: x, target embedding e_t, model f
eps=0.062, T=5, K=20
mu_in=0.9, mu_out=0.5, gamma=2.0, kesai=0.15
adv <- x; g_prev <- 0; g_pgia[K] <- 0

for t = 1..T:
  grads <- []
  for k = 1..K:
    noise <- U(-gamma*eps, gamma*eps)
    x_near <- adv + noise
    x_min <- x_near + kesai*eps*g_pgia[k]
    grad_k <- d/dx_min AttackLoss(f(x_min), e_t)
    grads.append(grad_k)

  g_pgia <- normalize(grads) - mu_in * g_pgia # inner momentum
  grad <- mean(grads) # expectation
over
  grad <- normalize(grad) # the
neighborhood
  grad <- grad + mu_out * g_prev # outer momentum
  g_prev <- grad

  adv <- adv + (eps/T) * sign(grad)
  adv <- clip(adv, x-eps, x+eps); clip(adv,-1,1)
return adv
```

## Hyperparameters Used

| Parameter                       | Value               |
|---------------------------------|---------------------|
| $\epsilon$ ( $L^\infty$ budget) | 0.062               |
| Iterations T                    | 5 (shared protocol) |
| Inner samples K                 | 20 (paper default)  |
| Inner momentum $\mu$            | 0.9                 |
| Outer momentum $\mu$            | 0.5                 |
| Neighborhood radius $\gamma$    | 2.0                 |
| Step-to-flat $\kappa$ (kesai)   | 0.15                |

**T=5 and  $\epsilon=0.062$**  are fixed by the repo's shared evaluation protocol. Every attack here is compared under the same budget.

**K,  $\gamma$ ,  $\kappa$ ,  $\mu$ 's** are the paper's defaults; not yet independently tuned for face verification specifically. This is a natural next step.

# Adapting MEF to Face Verification

## Two Attack Goals

- Impersonation:  $\text{loss} = \text{mean}(\cos)$
- Dodging:  $\text{loss} = \text{mean}(1 - \cos)$
- Same objective formulation as every other attack in the pipeline

## Pipeline Integration

- One surrogate per run: Facenet512, ArcFace, GhostFaceNet, or VGG-Face (via DeepFace)
- Evaluated cross-model on unseen victim models
- Same preprocessing,  $\epsilon$ , and T as the shared protocol

## What I Personally Verified

- Ran the full 4-surrogate  $\times$  4-victim sweep (360 evaluation cases) end-to-end myself
- Wrote the evaluation script: embed adversarial images under each victim model, merge against the repo's official clean-similarity baseline CSV, compute breach rate and impact
- Spot-checked the merged output row-by-row against raw similarity values before trusting any final number

# Evaluation Environment

1

## Scale

30 face pairs  $\times$  2 goals  $\times$  4 surrogates  $\times$  3 non-self victims = 360 evaluation cases

2

## Surrogate models

Facenet512, ArcFace, GhostFaceNet, VGG-Face — all via DeepFace framework

3

## Victim models

Facenet512, ArcFace, GhostFaceNet, VGG-Face (IR152 not yet wired in — see limitations)

4

## Metrics

Breach rate: % crossing the victim's threshold. Impact: mean cosine-similarity shift vs. clean baseline

## Reproducibility Note

Adversarial generation is stochastic (random neighborhood sampling), so exact breach-rate figures can vary slightly run to run — following the repo's own baseline note, small deviations are expected.

All figures in this deck come from a single, fully-logged run across all 4 surrogate models, merged against the repo's official clean-similarity CSV — not partial results.

# Results

| Metric                             | MEF (this work)          |
|------------------------------------|--------------------------|
| Overall breach rate                | 38.89% (140 / 360 cases) |
| Mean embedding impact              | 0.1887                   |
| Dodging breach rate                | 51.67%                   |
| Impersonation breach rate          | 26.11%                   |
| Best vanilla baseline (SI-NI-FGSM) | 29.17%                   |
| Gain over best vanilla baseline    | +9.72 pp                 |

*Note: MEF's figures cover 4 of 5 victim models. IR152 requires a separate PyTorch loader not yet wired into the evaluation script.*

# Cross-Model Transferability

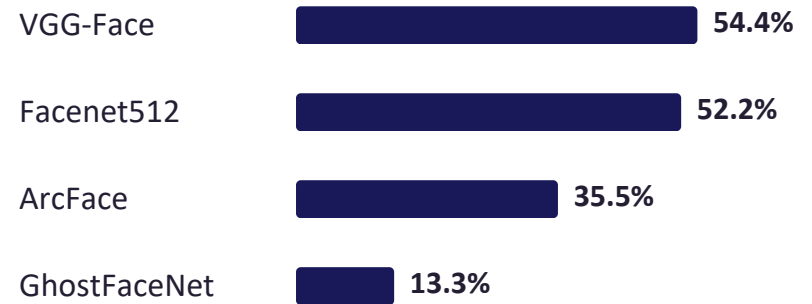
Row = surrogate model. Column = victim model. Breach rate (%); diagonal (self-transfer) excluded.

| Surrogate ↓ / Victim → | Facenet512 | ArcFace | GhostFaceNet | VGG-Face |
|------------------------|------------|---------|--------------|----------|
| Facenet512             | —          | 53.3    | 43.3         | 60.0     |
| ArcFace                | 30.0       | —       | 43.3         | 33.3     |
| GhostFaceNet           | 6.7        | 23.3    | —            | 10.0     |
| VGG-Face               | 60.0       | 63.3    | 40.0         | —        |

## Average Breach by Victim



## Average Breach by Surrogate



## Takeaway

- Surrogate choice matters as much as the attack algorithm
- GhostFaceNet's lightweight architecture likely shares less gradient structure with the heavier surrogates

## Best / Worst Surrogates

- VGG-Face: 54.4% avg breach (best)
- Facenet512: 52.2% avg breach
- ArcFace: 35.5% avg breach
- GhostFaceNet: 13.3% avg breach (weakest)

# Limitations & Next Steps

- 1 IR152 victim not yet included — needs a separate PyTorch loader (core/ir152.py), not yet wired into the evaluation script
- 2 Hyperparameters ( $K$ ,  $\gamma$ ,  $\kappa$ , momentum terms) are the paper's published defaults — not independently tuned for face verification
- 3 Single run, CPU-only — inner-loop sampling is stochastic; a multi-seed run would give a confidence interval instead of a point estimate
- 4 Next step: wire up IR152 for full 5-victim parity, then ablate  $K$  and  $\gamma$  for this specific setting